Caching Expensive Calculations

Memoization stores results by argument, so the second call is a lookup instead of a recomputation.

The problem: repeated work

```
calls = 0
def fib(n):
    global calls
    calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(20), calls)
```

OUTPUT

```
6765 21891
```

A hand-made memoize decorator

```
from functools import wraps

def memoize(fn):
    cache = {}
    @wraps(fn)
    def wrapper(*args):
        # miss: compute + store
        if args not in cache:
            cache[args] = fn(*args)
        # hit: just look it up
        return cache[args]
    return wrapper

calls = 0
@memoize
def fib(n):
    global calls
    calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(20), calls)
```

OUTPUT

```
6765 21
```

The standard library way

```
from functools import lru_cache, cache

# keep the 128 most recent results
@lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(30))
info = fib.cache_info()
print(info.hits, info.misses, info.currsize)
fib.cache_clear()
```

OUTPUT

```
832040
28 31 31
```

Tool	Notes
`@lru_cache(maxsize=N)`	evicts least-recently-used beyond N entries
`@cache` (3.9+)	= `lru_cache(maxsize=None)`, unbounded
`f.cache_info()`	hits, misses, maxsize, currsize
`f.cache_clear()`	empty the cache
`@cached_property`	compute once per object, store as attribute

Arguments must be hashable

```
@cache
def total(xs):
    return sum(xs)

try:
    total([1, 2])
except TypeError as e:
    print(e)
print(total((1, 2)))          # a tuple works
```

OUTPUT

```
unhashable type: 'list'
3
```

Rules for safe caching

- Cache only **pure** functions (same arguments → same result). Never cache `time.time()`, DB reads that change, or I/O.
- Unbounded caches grow forever – set `maxsize` for long-running programs.
- No built-in expiry (TTL); use `cachetools.TTLCache` or store timestamps.
- On methods, `lru_cache` keeps `self` alive (memory leak) – prefer `cached_property`.
- A cached mutable result is shared – callers who mutate it change the cache.

TIP Check the payoff with `cache_info()`: lots of hits = good fit; all misses = remove the cache.